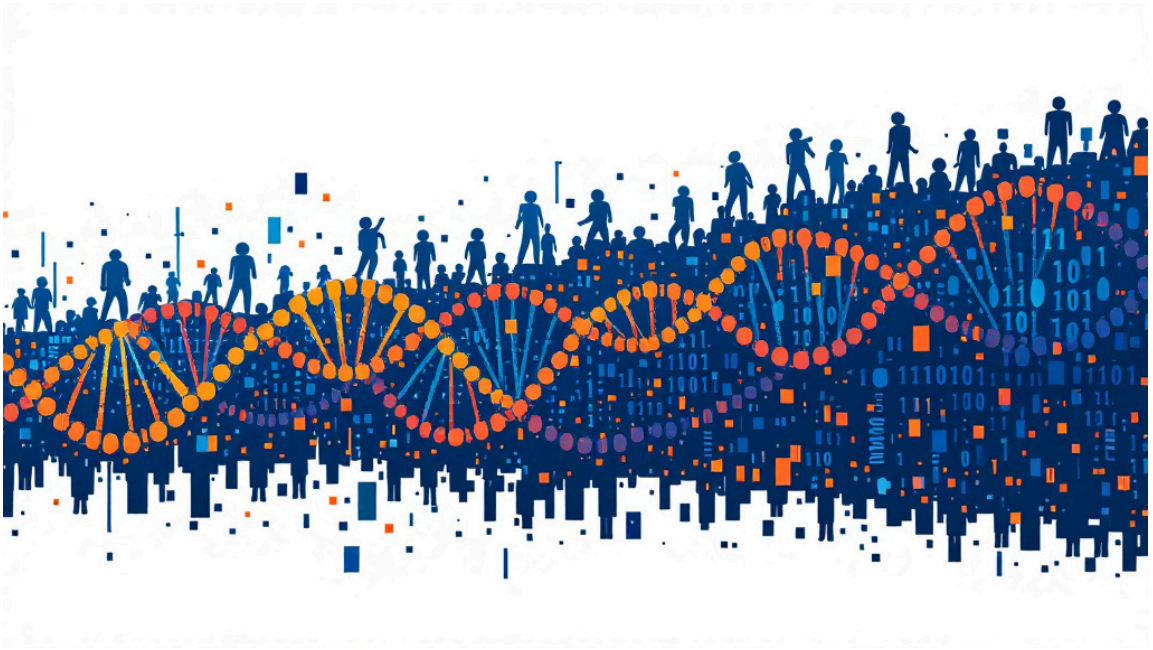
LABORATORIUM 2

Algorytmy genetyczne

Przedmiot	Sztuczna inteligencja
Forma	Laboratorium
Czas	4 h dydaktyczne (190 min), 03.11.2026
Grupy	INLS5_SI
Prowadzący	mgr inż. Szymon Guzik

Cele zajęć

- Opisać problem optymalizacyjny jako chromosom i funkcję oceny (fitness).
- Zaimplementować selekcję, krzyżowanie, mutację i elitaryzm.
- Rozwiązać problem plecakowy (kodowanie binarne) i komiwojażera (kodowanie permutacyjne).
- Zbadać wpływ parametrów (populacja, prawdopodobieństwo mutacji) na wynik.
- Przedstawić prowadzącemu postęp projektu (etap 1).



Ewolucja rozwiązań — populacja poprawia się z pokolenia na pokolenie

1. Plan zajęć (190 minut)

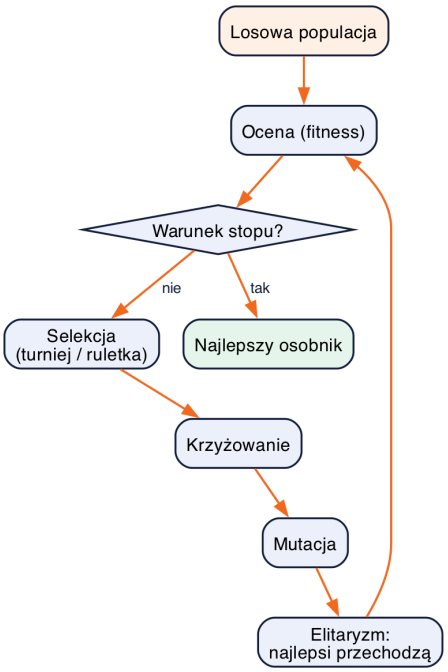
Czas	Co robimy
0–25 min	Idea algorytmów ewolucyjnych, pojęcia: osobnik, chromosom, gen, fitness

Czas	Co robimy
25–75 min	Problem plecakowy — przykład 1, zadania 1–2
75–85 min	Przerwa
85–135 min	Problem komiwojażera — przykład 2, zadanie 3
135–160 min	Badanie parametrów — zadanie 4
160–190 min	Konsultacje projektów — etap 1 (temat, repozytorium, plan)

2. Teoria

> **Idea**
>
> Algorytm genetyczny (John Holland, 1975) naśladuje dobór naturalny. Zamiast szukać rozwiązania wprost, utrzymujemy **populację** rozwiązań kandydujących. Lepsze (według **funkcji oceny**) częściej się rozmnażają, potomstwo dziedziczy fragmenty rodziców (**krzyżowanie**) i czasem losowo się zmienia (**mutacja**). Po wielu pokoleniach populacja skupia się wokół dobrych rozwiązań. Metoda nie gwarantuje optimum, ale radzi sobie tam, gdzie pełne przeszukanie jest niewykonalne.

Pojęcie	Znaczenie	W problemie plecakowym
Osobnik / chromosom	Jedno rozwiązanie kandydujące	Lista 12 bitów
Gen	Jedna pozycja chromosomu	Czy bierzemy i-ty przedmiot
Funkcja oceny (fitness)	Jak dobre jest rozwiązanie	Wartość przedmiotów, 0 przy przekroczeniu wagi
Selekcja	Wybór rodziców	Turniej: najlepszy z 3 losowych
Krzyżowanie	Łączenie rodziców	Cięcie w losowym punkcie i zamiana końcówek
Mutacja	Losowa zmiana	Odwrócenie bitu z p = 0,05
Elitaryzm	Ochrona najlepszych	2 najlepszych przechodzi bez zmian



Pętla algorytmu genetycznego

2.1. Kodowanie rozwiązań

Najważniejsza decyzja projektowa to **kodowanie**. Musi pozwalać zapisać każde sensowne rozwiązanie, a operatory genetyczne nie powinny tworzyć rozwiązań niepoprawnych. Dla plecaka wystarczy ciąg bitów. Dla komiwojażera chromosom jest **permutacją** miast — zwykłe krzyżowanie jednopunktowe dałoby trasy z powtórzonymi miastami, dlatego używamy krzyżowania **OX** (*Order Crossover*).

Plecak — kodowanie binarne	1	1	0	0	0	1	0	1	1	0	1	1

TSP — kodowanie permutacyjne	Gdańsk	Gdynia	Kartuzy	Lębork	…

Chromosom binarny (gen = weź / nie bierz przedmiotu) i permutacyjny (kolejność miast)

2.2. Selekcja

- **Ruletka** — prawdopodobieństwo wyboru proporcjonalne do oceny; wrażliwa na skalę funkcji oceny.
- **Turniej** — losujemy k osobników i bierzemy najlepszego; k steruje presją selekcyjną.
- **Rankingowa** — liczy się miejsce w rankingu, a nie wartość oceny.

Ograniczenia

Rozwiązania niespełniające ograniczeń (plecak za ciężki) można: odrzucić (kara śmierci — ocena 0), ukarać proporcjonalnie do przekroczenia albo **naprawić** (usuwać przedmioty, aż waga się zmieści). Kara śmierci jest najprostsza, ale przy ciasnych ograniczeniach populacja może utknąć.

3. Przykład 1 — problem plecakowy

Mamy 12 przedmiotów i plecak o nośności 7 kg. Szukamy zestawu o największej wartości. Wszystkich kombinacji jest $2^{12} = 4096$, więc tu możemy sprawdzić wynik pełnym przeglądem: **optimum to 8740 zł** — algorytm genetyczny znalazł je w kilkunastu pokoleniach. Przy 100 przedmiotach kombinacji byłoby 2^{100} i pełny przegląd przestaje być możliwy.

plecak_ga.py

```python
"""Algorytm genetyczny dla problemu plecakowego 0/1 (kodowanie binarne, selekcja turniejowa, elitaryzm)."""
import random

PRZEDMIOTY = [  # (nazwa, waga kg, wartość zł)
    ("laptop", 2.2, 3000), ("aparat", 0.9, 1800), ("książka", 0.6, 60), ("namiot", 3.5, 400),
    ("śpiwór", 1.8, 250), ("powerbank", 0.4, 150), ("kuchenka", 1.2, 200), ("lornetka", 0.8, 500),
    ("dron", 1.4, 2500), ("gitara", 3.0, 900), ("słuchawki", 0.3, 700), ("termos", 0.7, 90),
]
LIMIT = 7.0
random.seed(42)


def ocena(osobnik):
    waga = sum(p[1] for p, g in zip(PRZEDMIOTY, osobnik) if g)
    wartosc = sum(p[2] for p, g in zip(PRZEDMIOTY, osobnik) if g)
    return wartosc if waga <= LIMIT else 0          # kara śmierci za przekroczenie limitu


def turniej(populacja, k=3):
    return max(random.sample(populacja, k), key=ocena)


def krzyzowanie(a, b):
    punkt = random.randint(1, len(a) - 1)           # krzyżowanie jednopunktowe
    return a[:punkt] + b[punkt:], b[:punkt] + a[punkt:]


def mutacja(osobnik, p=0.05):
    return [1 - g if random.random() < p else g for g in osobnik]


def ga(pokolenia=60, rozmiar=40, elita=2):
    populacja = [[random.randint(0, 1) for _ in PRZEDMIOTY] for _ in range(rozmiar)]
    for nr in range(pokolenia):
        populacja.sort(key=ocena, reverse=True)
        if nr % 15 == 0:
            print(f"pokolenie {nr:2}: najlepszy = {ocena(populacja[0])} zł")
        nowa = populacja[:elita]                    # elitaryzm: najlepsi przechodzą bez zmian
        while len(nowa) < rozmiar:
            d1, d2 = krzyzowanie(turniej(populacja), turniej(populacja))
            nowa += [mutacja(d1), mutacja(d2)]
        populacja = nowa[:rozmiar]
    return max(populacja, key=ocena)


if __name__ == "__main__":
    najlepszy = ga()
    wybrane = [p[0] for p, g in zip(PRZEDMIOTY, najlepszy) if g]
    waga = sum(p[1] for p, g in zip(PRZEDMIOTY, najlepszy) if g)
    print("wynik:", ocena(najlepszy), "zł, waga", round(waga, 1), "kg ->", ", ".join(wybrane))

```

kod/plecak_ga.py

```
$ python3 -B kod/plecak_ga.py
pokolenie  0: najlepszy = 8250 zł
pokolenie 15: najlepszy = 8740 zł
pokolenie 30: najlepszy = 8740 zł
pokolenie 45: najlepszy = 8740 zł
wynik: 8740 zł, waga 6.7 kg -> laptop, aparat, powerbank, lornetka, dron, słuchawki, termos
```

Zadanie 1 (20 min)

Zamień **karę śmierci** na **naprawę**: jeśli plecak jest za ciężki, usuwaj przedmiot o najmniejszym stosunku wartość/waga, aż waga się zmieści. Porównaj, w którym pokoleniu obie wersje po raz pierwszy znajdują 8740 zł (uruchom po 10 razy z różnym `random.seed`).

Zadanie 2 (15 min)

Zaimplementuj **selekcję ruletkową** i porównaj z turniejową. Kiedy ruletka zawodzi? (Wskazówka: co się dzieje, gdy prawie wszystkie osobniki mają ocenę 0?)

4. Przykład 2 — problem komiwojażera (TSP)

Kurier ma odwiedzić 10 miast Pomorza i wrócić do Gdańska najkrótszą trasą. Tras jest 9!/2 = 181 440. Chromosom to permutacja miast, ocena to długość trasy (mniejsza = lepsza).

tsp_ga.py

```python
"""Algorytm genetyczny dla problemu komiwojażera (kodowanie permutacyjne, krzyżowanie OX, mutacja zamiany)."""
import math
import random

MIASTA = {"Gdańsk": (54.35, 18.65), "Gdynia": (54.52, 18.53), "Słupsk": (54.46, 17.03),
          "Kościerzyna": (54.12, 17.98), "Tczew": (54.09, 18.78), "Malbork": (54.04, 19.03),
          "Elbląg": (54.16, 19.40), "Kartuzy": (54.33, 18.20), "Lębork": (54.54, 17.75),
          "Chojnice": (53.70, 17.56)}
NAZWY = list(MIASTA)
random.seed(7)


def odl(a, b):
    (x1, y1), (x2, y2) = MIASTA[a], MIASTA[b]
    return math.hypot((x1 - x2) * 111, (y1 - y2) * 65)   # przybliżenie km na tej szerokości


def dlugosc(trasa):
    return sum(odl(trasa[i], trasa[(i + 1) % len(trasa)]) for i in range(len(trasa)))


def ox(a, b):
    """Order Crossover: fragment z rodzica A, reszta miast w kolejności z rodzica B."""
    i, j = sorted(random.sample(range(len(a)), 2))
    srodek = a[i:j]
    reszta = [m for m in b if m not in srodek]
    return reszta[:i] + srodek + reszta[i:]


def mutacja(t, p=0.2):
    t = t[:]
    if random.random() < p:
        i, j = random.sample(range(len(t)), 2)
        t[i], t[j] = t[j], t[i]
    return t


def ga(pokolenia=300, rozmiar=60):
    pop = [random.sample(NAZWY, len(NAZWY)) for _ in range(rozmiar)]
    start = min(map(dlugosc, pop))
    for _ in range(pokolenia):
        pop.sort(key=dlugosc)
        nowa = pop[:4]
        while len(nowa) < rozmiar:
            a, b = (min(random.sample(pop, 4), key=dlugosc) for _ in range(2))
            nowa.append(mutacja(ox(a, b)))
        pop = nowa
    return start, min(pop, key=dlugosc)


if __name__ == "__main__":
    start, best = ga()
    print(f"najlepsza losowa trasa: {start:.0f} km")
    print(f"po ewolucji:            {dlugosc(best):.0f} km")
    print(" -> ".join(best + [best[0]]))

```

kod/tsp_ga.py

```
$ python3 -B kod/tsp_ga.py
najlepsza losowa trasa: 552 km
po ewolucji:            431 km
Gdańsk -> Gdynia -> Kartuzy -> Lębork -> Słupsk -> Chojnice -> Kościerzyna -> Tczew ->
Malbork -> Elbląg -> Gdańsk
```

Zadanie 3 (30 min)

1. Dodaj mutację **inwersji** (odwrócenie fragmentu trasy) i porównaj ją z zamianą dwóch miast.
2. Narysuj najlepszą trasę biblioteką `matplotlib` (współrzędne z `MIASTA`) i zapisz wykres do PNG.
3. Wypisz długość najlepszej trasy co 50 pokoleń i narysuj krzywą zbieżności.

5. Badanie parametrów

Zadanie 4 (25 min)

Dla TSP przygotuj tabelę: rozmiar populacji $\in \{20, 60, 150\}$ × prawdopodobieństwo mutacji $\in \{0{,}01; 0{,}2; 0{,}6\}$. Dla każdej kombinacji uruchom algorytm 5 razy i zapisz średnią długość trasy. Które ustawienie jest najlepsze i dlaczego zbyt duża mutacja szkodzi? Wnioski zapisz w 3–4 zdaniach.

6. Projekt — etap 1

Konsultacja (30 min)

Każdy zespół pokazuje: repozytorium z `README.md`, opis problemu, wybrane metody SI i plan pracy na kolejne tygodnie. Jeśli w projekcie jest optymalizacja (rozkład zajęć, trasa, dobór parametrów), dzisiejszy algorytm genetyczny może być jednym z jego modułów.

7. Literatura

- G. Bonaccorso, *Algorytmy uczenia maszynowego*, Helion 2019.
- D. Rutkowska, M. Piliński, L. Rutkowski, *Sieci neuronowe, algorytmy genetyczne i systemy rozmyte*, PWN 1997.
- S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach* — rozdz. o przeszukiwaniu lokalnym.